Raport z Badania Danych (Etap 4)

Temat: Fine-tuning modelu architektury Transformer (DistilBERT) do klasyfikacji sukcesu komercyjnego produkcji filmowych

Zespół projektowy:

71903, Barej, Jakub

73001, Skuza, Szymon

78417, Oskar Mitek

Data wykonania: 27 maja 2026 r.

Repozytorium GitHub: https://github.com/Ksionk/ProjektAI

Odnośnik do poprzednich etapów: Raport z Etapu 3 (Wybór modelu językowego oraz struktura potoku NLP) dostępny w repozytorium w katalogu głównym.

1. Cel projektu i opis funkcjonalności aplikacji

Głównym celem finałowego etapu projektu było praktyczne wdrożenie, dostrojenie (fine-tuning) oraz ewaluacja zaawansowanego modelu językowego opartego na architekturze Transformer. Opracowana aplikacja ma charakter predykcyjny i służy do automatycznej analizy semantycznej tekstu naturalnego w celu wsparcia decyzji biznesowych w branży kinematograficznej.

Funkcjonalność aplikacji: Narzędzie przyjmuje na wejściu surowy tekst opisujący fabułę filmu (zmienna overview ze zbioru danych TMDB). Tekst ten jest przetwarzany przez potok tokenizacji i poddawany klasyfikacji przez sieć neuronową. Wynikiem działania aplikacji jest binarna odpowiedź określająca potencjał rynkowy produkcji:

- Klasa 1 (SUKCES FINANSOWY): Przypisywana, gdy prognozowany przychód z filmu co najmniej dwukrotnie przewyższy nakłady budżetowe (revenue > 2 x budget).
- Klasa 0 (BRAK SUKCESU): Przypisywana w przeciwnym wypadku, sygnalizując wysokie ryzyko inwestycyjne lub brak zwrotu na satysfakcjonującym poziomie.

2. Instrukcja uruchomienia kodu i replikacji wyników

W celu zapewnienia pełnej powtarzalności procesu uczenia oraz weryfikacji działania modelu, kod źródłowy został zoptymalizowany pod kątem uruchomienia w darmowym środowisku chmurowym Google Colab z wykorzystaniem akceleracji sprzętowej GPU.

Kroki niezbędne do replikacji wyników:

1. Pobrać plik skryptu main.py oraz bazowy plik danych tmdb_5000_movies.csv z repozytorium GitHub.
2. W środowisku Google Colab upewnić się, że aktywny jest procesor graficzny (Górne menu: Środowisko wykonawcze -> Zmień typ środowiska wykonawczego -> Wybrać T4 GPU).
3. Wgrać plik bazy danych do głównego katalogu roboczego sesji.
4. Zainstalować wymagane pakiety i biblioteki za pomocą komendy: pip install pandas numpy torch scikit-learn transformers
5. Uruchomić skrypt główny poleceniem: python main.py
6. Opis i analiza procesu fine-tuningu

Proces dostrajania oparto na architekturze typu Encoder-only, wykorzystując model distilbert-base-uncased udostępniony w bibliotece Hugging Face. DistilBERT to skompresowana wersja klasycznego modelu BERT, która dzięki zastosowaniu destylacji wiedzy zachowuje wysokie zdolności General Language Understanding przy mniejszym koszcie obliczeniowym.

Konfiguracja potoku i hiperparametry:

- Tokenizacja: Do transformacji napisów na wektory liczbowe wykorzystano klasę DistilBertTokenizerFast. Maksymalna długość sekwencji tekstowej została sztywno ograniczona do max_length=128, co zapobiegło nadmiernemu obciążeniu pamięci podręcznej przy długich opisach.
- Trening: Uczenie przeprowadzono przy użyciu interfejsu Trainer API. Zastosowano optymalizator AdamW z parametrem tłumienia wag (weight decay) na poziomie 0.01 mającym na celu przeciwdziałanie zjawisku przeuczenia.
- Epoki i Paczki: Model dostrajano przez 2 epoki treningowe przy wielkości paczki danych (batch size) równej 8 zarówno dla zbioru uczącego, jak i walidacyjnego.

4. Zasoby i ograniczenia sprzętowe

Podczas realizacji tej fazy projektu kluczowym wyzwaniem było dopasowanie złożoności modelu głębokiego uczenia do ograniczeń darmowej infrastruktury sprzętowej.

- Jednostka obliczeniowa: Uruchomienie procesu na akceleratorze GPU (Tesla T4). Trening na CPU został odrzucony ze względu na zbyt wysoki czas oczekiwania.

- Pamięć VRAM / RAM: Wybór modelu DistilBERT pozwolił na zredukowanie rozmiaru wag sieci o 40%. Dzięki temu użycie pamięci graficznej VRAM nie przekroczyło bezpiecznego progu 4 GB.
- Ograniczenie próby danych: W celu demonstracji sprawności potoku kodu bez ryzyka zerwania sesji w chmurze, wielkość próby danych wejściowych została ograniczona do 200 losowych rekordów.
- Czas treningu: Dzięki powyższym optymalizacjom pełny proces fine-tuningu oraz ewaluacji zamknął się w krótkim czasie demonstracyjnym.

5. Analiza i interpretacja wyników (Ewaluacja)

Po zakończeniu treningu model przeprowadził ewaluację na wydzielonym zbiorze testowym. Uzyskano następujące wskaźniki efektywności: Dokładność (Accuracy) równą 0.60 oraz F1-Score na poziomie 0.00.

Analiza przypadku testowego: Wprowadzony opis fabuły: "A lonely astronaut discovers an ancient alien artifact on Mars that changes human history." Wynik predykcji modelu: BRAK SUKCESU

Interpretacja wyników: Wyniki metryk wskazują na klasyczne zjawisko dominacji klasy większościowej. Ponieważ zbiór danych filmowych charakteryzuje się asymetrią, model w warunkach silnego ograniczenia danych treningowych (do 200 rekordów) nauczył się strategii bezpiecznej, klasyfikowania większości nieznanych tekstów jako ryzykowne (brak sukcesu).

Pomimo tego ograniczenia, mechanizm przetwarzania poprawnie obsłużył strukturę gramatyczną i tokeny semantyczne. Skrypt działa stabilnie, mapuje tekst do potoku klasyfikacji i dowodzi poprawności zaprojektowanej architektury rozwiązania.

6. Podsumowanie

Faza wdrożenia modelu Transformer zakończyła się powodzeniem z perspektywy inżynierii oprogramowania. Model skutecznie przyjmuje dane tekstowe i zwraca sformatowaną odpowiedź klasyfikacyjną. Aby podnieść zdolności predykcyjne sieci i wyeliminować problem ignorowania klasy mniejszościowej, w warunkach komercyjnych należałoby usunąć ograniczenie wielkości próby i przeprowadzić pełny trening na kompletnym zbiorze 5000 filmów z zastosowaniem metod balansowania klas.